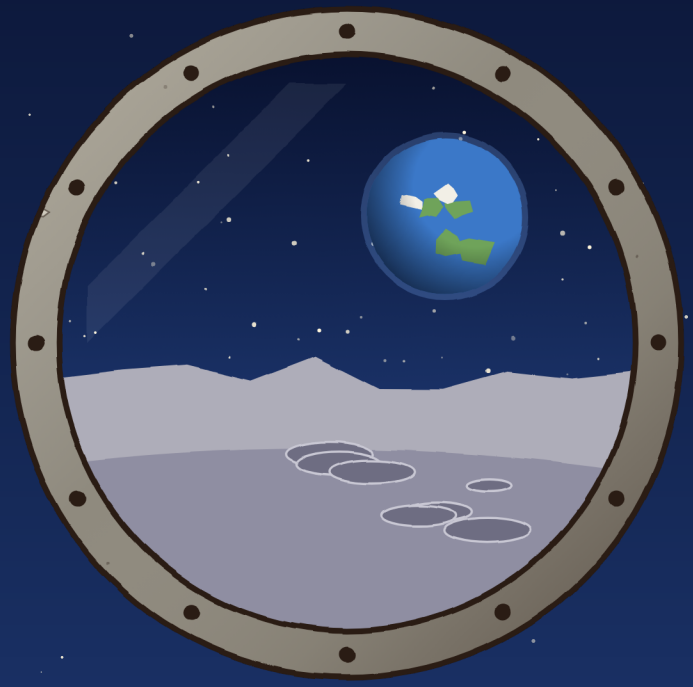


POISONED DATASET

Draft 0.1 · rules and art are working hypotheses, not measured results.
Source, helper and printable kit: github.com/haydn-s/poisoned-dataset-game · Apache 2.0

POISONED DATASET

Who changed the labels?

A two-player game of data poisoning and scarce audits

2 PLAYERS · TARGET 15–20 MINUTES · DRAFT 0.1

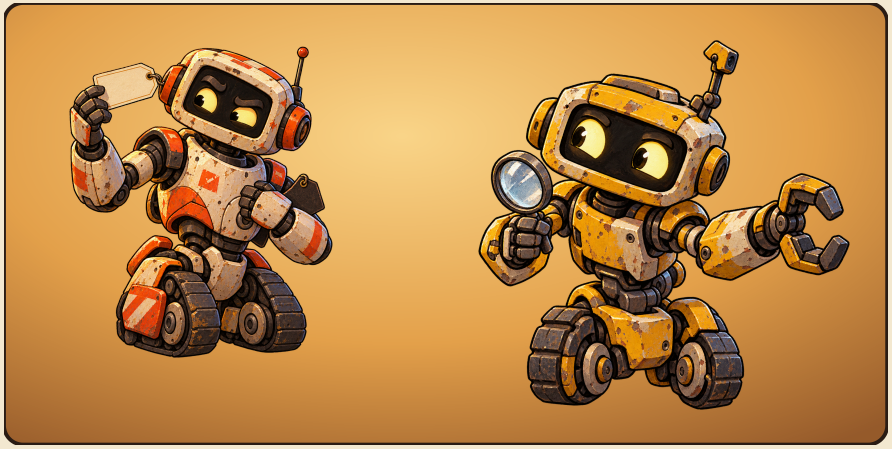


THE STORY

Your mission

You work for rival companies on a moon colony. The **defender** trains a supply-sorting robot to tag cargo **STORE** or **RECYCLE**. The **attacker** secretly changes a few training labels to make the robot learn the wrong rule.

Play two rounds and switch roles after the first, so each player defends once. The better defender wins.



What the game is about

Machine-learning systems learn from labelled examples. If someone can change the labels, the system can learn a rule that looks right on its training data and fails in use. Checking every label is expensive, so defenders choose which few to verify.

> This is a deliberately tiny learner. The game explains label poisoning and trusted audits; it does not show that any real ML system is robust, and it is not a measured teaching intervention.

AT A GLANCE

Quick reference

ONE ROUND QUICK REFERENCE

1. **Attacker secretly flips up to 4 tags.**
2. **Defender counts each rule's errors. Looking is free.**
3. **Audit: pay a token, turn that cargo's reference card, set the tag to match.**
4. **Lock the fewest-error rule. Ties: red, round, striped.**
5. **Reveal the truth card. 1 point per test card the locked rule gets right.**
6. **Check every reference card against the truth; switch roles.**

Budgets in draft 0.1

24 training cards per round · up to 4 flips · 4 audit tokens · 8 test cards · ties go red, then round, then striped.

Playtesting?

Share the short feedback form in `playtest.md` after both rounds. Note which text, symbols or pieces were hard to read or handle.

TALK IT OVER

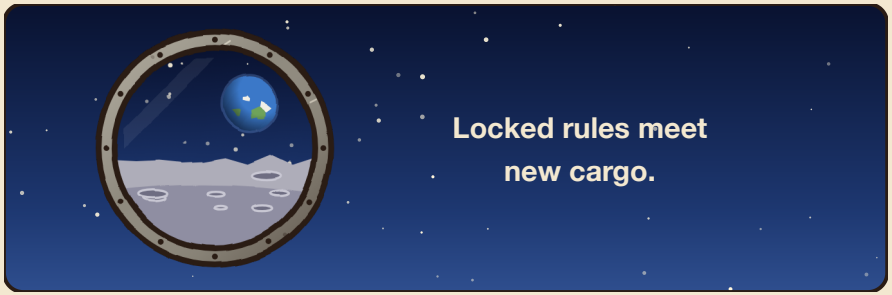
Debrief

The most interesting part of a match is often the reveal. Use these questions after both rounds:

1. Which flipped tags changed the competition between rules, and why?
2. Did finding a wrong tag immediately repair the score? What would one fewer audit have changed?
3. Could an attacker flip a whole group of look-alike cards so no disagreement remains?
4. What makes the reference cards trustworthy here? What changes if checks cost more or can be wrong?
5. What does this tiny rule library help explain, and what does it leave out about real ML systems?

What this models, and what it does not

The game models label poisoning before training and a fixed budget of perfectly trustworthy audits against a three-rule learner. Real systems have far more features, noisier references and attackers who adapt. Treat the discussion as feedback on the game, not proof that anyone learned something.



IN THE BOX

Components

Everything is printed once and reused. During a round only two things are secret: which truth card the attacker drew, and which tags they flipped.



96 training cards
four decks, A–D, of 24



8 test cards
one per cargo type, every round



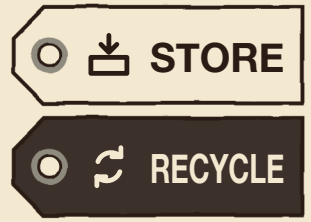
3 truth cards
one drawn in secret each round



16 reference cards
a STORE and a RECYCLE for each cargo type



3 rule cards
red, round, striped + tie order



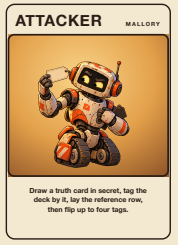
27 fold-over tags
STORE one side, RECYCLE the other



4 audit tokens
AUDIT side, then VERIFIED side



Score card
plus 2 quick references



2 role cards
attacker and defender

> Keep the unused truth and reference cards face down in the box during a round.

READING THE CARDS

Cargo and labels

Every training card is red or blue, round or square, and striped or plain. Features never change. The label is a separate hang-tag that says **STORE** or **RECYCLE**.



1. Card ID and deck. Audits and scores name cards by ID.
2. The cargo. Three features: colour, shape and pattern.
3. The manifest spells out each feature, so colour is never the only cue.
4. Hang the label tag here. The label is separate from the cargo.

Flip the tag, flip the label

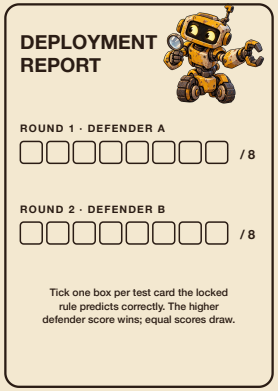
Each tag is folded so one side reads STORE and the other RECYCLE. Changing a label means turning its tag over; that is all an attacker may do.



AFTER TWO ROUNDS

Winning

Compare the scores the two players earned as **defender**. The higher score wins; equal scores are a draw. Both players have the same four-flip and four-audit budgets, but different scenarios are not guaranteed to be equally hard.



> With three rules and an eight-card test deck that covers every combination, a wrong rule always scores 4/8 and the true rule 8/8. Draws may be common. That is a known risk in draft 0.1: record draws rather than inventing a tiebreaker.

Why the test deck is separate

The robot is judged on new cargo it never trained on, just as a deployed system is. The test cards never join the training data, and their reference labels stay sealed until the rule is locked.



4 Lock the learned rule

When the defender stops or runs out of tokens, the rule with the fewest errors is locked, using the tie order. Unused tokens give no bonus.



5 Reveal and score

Reveal the truth card. For each of the eight test cards, place the tag the locked rule predicts; it scores a point if the truth card agrees. Then turn every reference card face up. If any disagrees with the truth card, the setup was wrong and the defender scores 8.



6 Review and switch

Reveal the flipped tags and the audits, and talk about what changed the learned rule. Return the truth and reference cards to the box, set out an unused deck, and switch roles.

The fine print

- A verified card cannot be audited again.
- An audit repairs only its own card. Identical-looking copies keep their tags until they are audited too.
- No free relabelling, deleting or adding cards in this version.
- The helper's audit hints are suggestions from visible evidence, not proof of poisoning.

HOW THE ROBOT LEARNS

Fewest errors wins

The robot can learn exactly one of three rules: **STORE if red**, **STORE if round**, or **STORE if striped**; anything else is RECYCLE. For each rule, count how many current tags it gets wrong. The rule with the fewest errors is the one the robot learns.

CARD	CURRENT TAG	RED	ROUND	STRIPED
C01 red, square, plain	STORE	✓	✗	✗
C02 blue, round, plain	RECYCLE	✓	✗	✓
C03 red, round, striped	STORE	✓	✓	✓
C04 blue, square, striped	RECYCLE	✓	✓	✗
ERRORS		0	2	2

Here red makes 0 errors, round 2 and striped 2, so the robot learns **red**. These tags alone do not prove red is the true rule.

Ties always break the same way

1 RED → 2 ROUND → 3 STRIPED

The defender cannot pick a favourite rule; the counts decide.

BEFORE EACH ROUND

Setup

1. **Lay out a deck you have not used this match:** its 24 training cards face up in ID order. Set the 8 test cards aside, and set out the rule cards and the score card.
2. **The defender looks away.** The attacker shuffles the three truth cards face down, draws one, reads it, and returns the other two to the box unseen.
3. **Tag by the truth.** Hang a tag on every training card, STORE side up wherever the truth card says STORE.
4. **Lay the reference row.** For each of the eight cargo types, lay face down the reference card whose label agrees with the truth. Return the other eight to the box unseen.
5. **The attacker poisons** (step 1 of the round).
6. **The defender returns** and takes four audit tokens, AUDIT side up.



> Features and rules are public, and so are the decks. The secrets are the drawn truth card and the flipped tags; the reference row is checked against the truth at the end.

Playing on a screen instead

`python3 -m poisoned_dataset play` runs the same rules with freshly generated decks and keeps the counts for you.

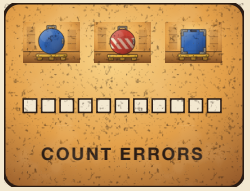
EACH ROUND

Playing a round



1 Secret poisoning

With the defender looking away, the attacker may turn over the tags of up to **four different** training cards. Nothing else changes: no features, cards, references or test cards. The attack is finished before auditing starts.



2 Inspect the data

The defender sees all 24 cards and their current tags. Count each rule's errors on the rule cards and announce the current winner. Looking, grouping and counting are free.



3 Spend audit tokens

Up to **four**, one at a time. Name an unaudited card and pay a token. Turn over the reference card for its cargo type and turn the card's tag to match, even if it was already right. Flip the token to **VERIFIED** and leave it on the card. Recount after every audit.